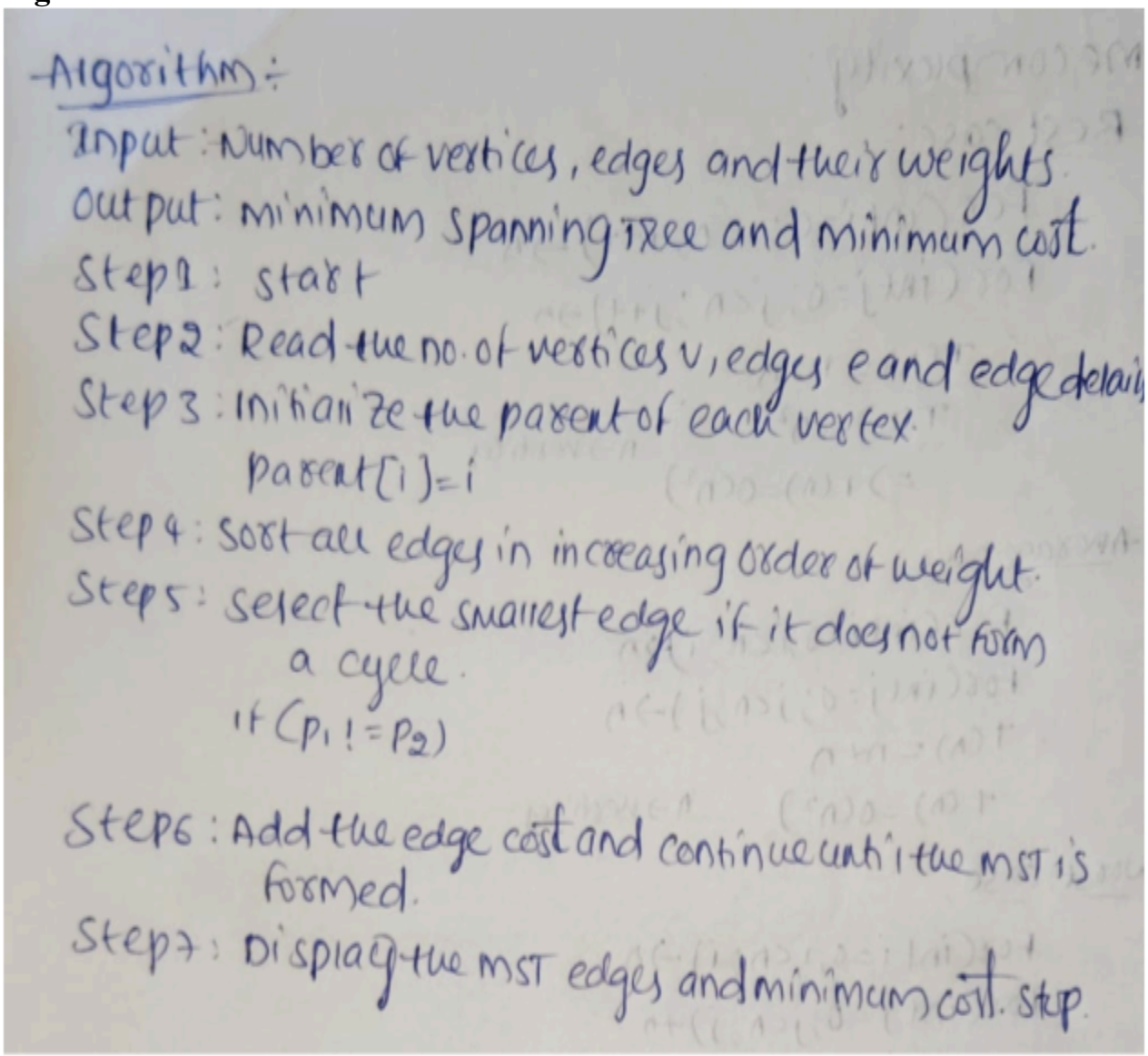
practical-10

Aim: To Implement Kruskal's algorithm

Kruskal's algorithm

Algorithm:



Program:

```
#include <iostream>
#include <algorithm>
using namespace std;
#define MAX 20

struct Edge

{
   int u, v, w;
};

Edge edge[MAX];
int parent[MAX];

// Find Parent
int find(int x)
{
   while(parent[x] != x)
      x = parent[x];

   return x;
}

// Union of Sets
void Union(int a, int b)
{
   parent[a] = b;
}

// Compare edges by weight
bool compare(Edge a, Edge b)
{
   return a.w < b.w;
}

int main()
{
   int v, e;
   int totalCost = 0;
```

```
cout << "Enter number of vertices: ";
cin >> v;

cout << "Enter number of edges: ";
cin >> e;
cout << "Enter Source Destination Weight:\n";

for(int i = 0; i < e; i++)

{
   cin >> edge[i].u
       >> edge[i].v
       >> edge[i].w;
}

// Initialize Parent
for(int i = 0; i < v; i++)
   parent[i] = i;

// Sort edges by weight
sort(edge, edge + e, compare);
cout << "\nEdges in Minimum Spanning Tree:\n";

for(int i = 0; i < e; i++)

{
   int p1 = find(edge[i].u);
   int p2 = find(edge[i].v);

   if(p1 != p2)
   {
      Union(p1, p2);

      cout << edge[i].u
           << " --> "
           << edge[i].v
           << " Cost = "
           << edge[i].w
           << endl;
```

```
            totalCost += edge[i].w;
        }
    }

    cout << "\nMinimum Cost = " << totalCost << endl;

    return 0;
}
```

Output:

```
Output

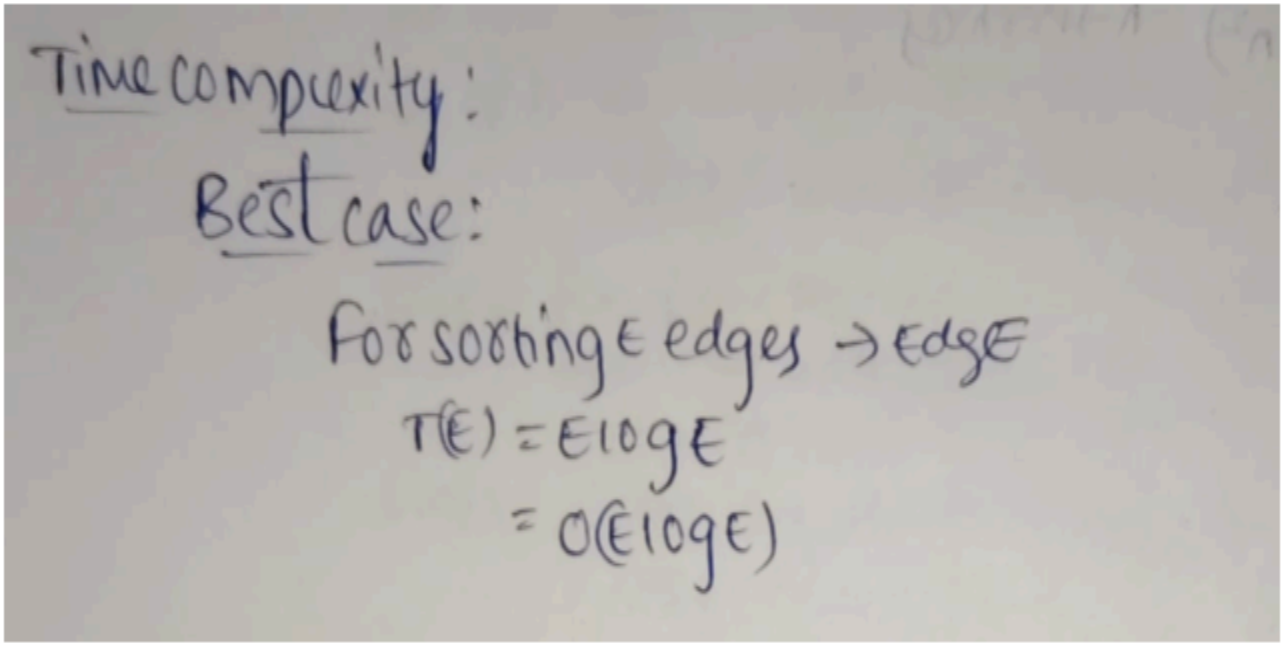
Enter number of vertices: 6
Enter number of edges: 8
Enter Source Destination Weight:
0 1 4
0 2 3
1 2 2
1 3 5
2 3 6
2 4 7
3 4 1
4 5 2

Edges in Minimum Spanning Tree:
3 --> 4  Cost = 1
1 --> 2  Cost = 2
4 --> 5  Cost = 2
0 --> 2  Cost = 3
1 --> 3  Cost = 5

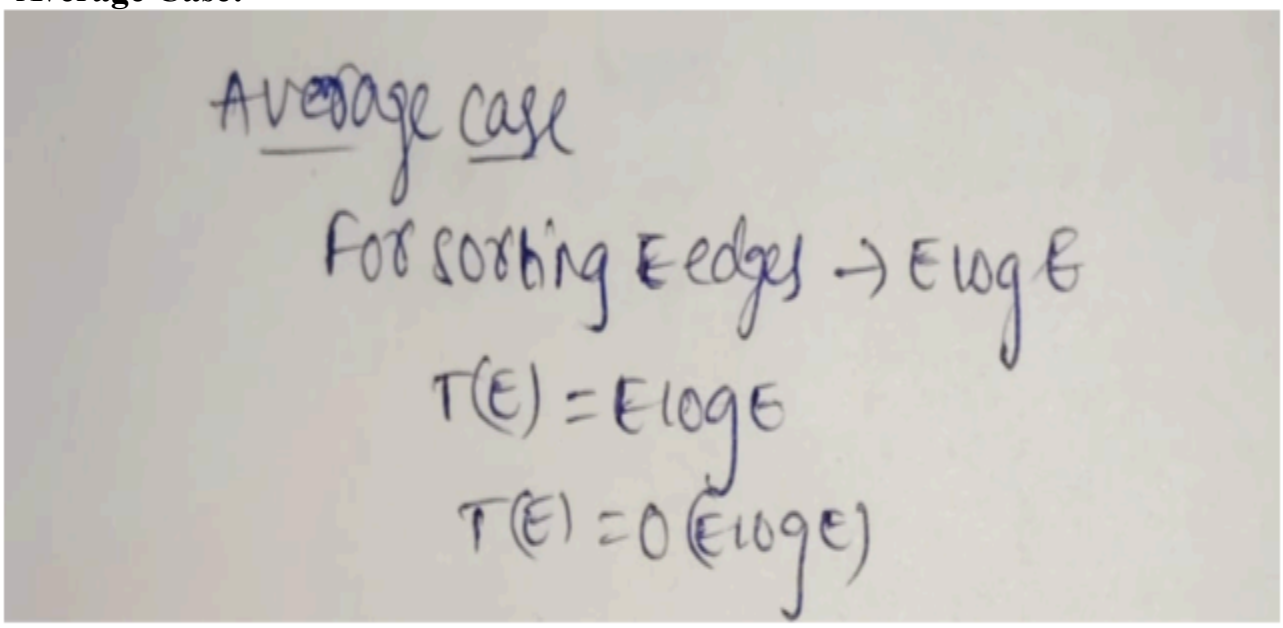
Minimum Cost = 13
```

Time Complexity:

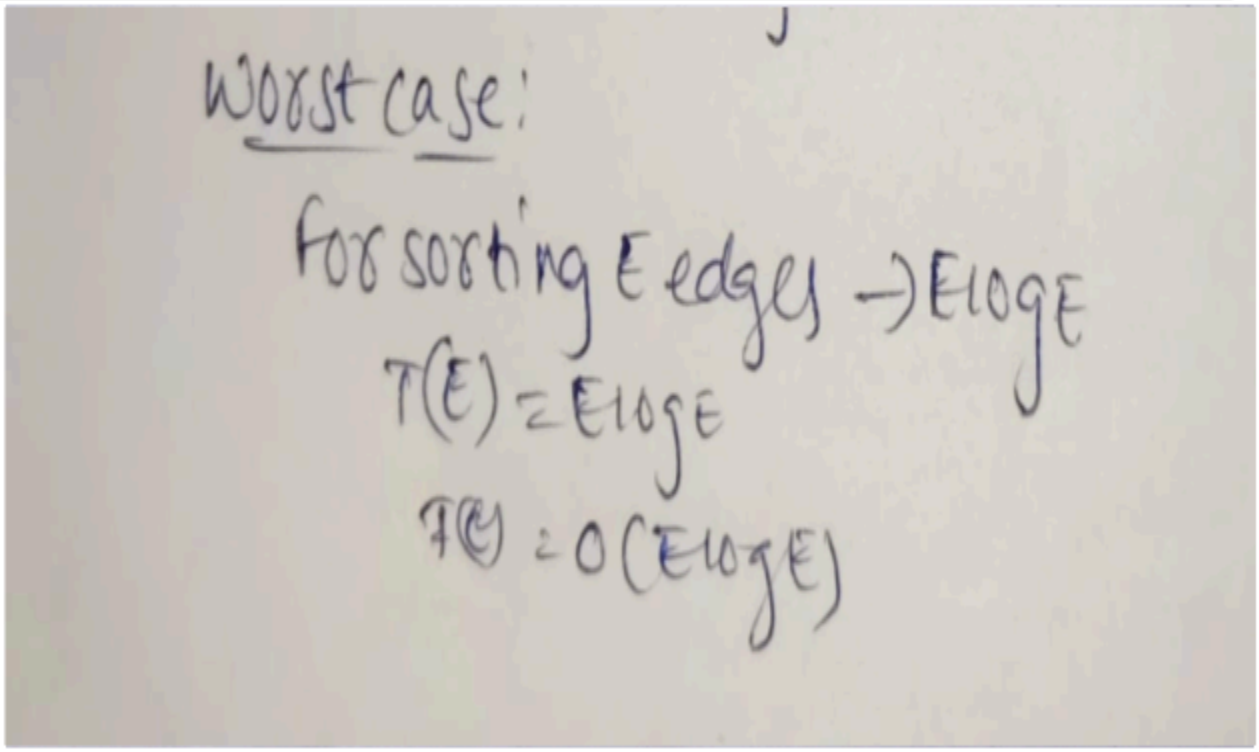
Best Case:



Average Case:



Worst Case:



Conclusion:

The implementation of Kruskal's Algorithm helps us understand how a Minimum Spanning Tree (MST) is constructed for a weighted graph. It selects edges in increasing order of their weights and avoids edges that form a cycle. For the given implementation using efficient sorting, Kruskal's Algorithm has a time complexity of O(e log e) in the best, average, and worst cases. Therefore, Kruskal's Algorithm is a useful technique for finding the minimum-cost spanning tree of a connected graph.